CS 422 Assignment 4: Maximum Concurrent Flow and Network Design

CS 422 project team

November 2026

1 Submission and code map

Repository: Assignment 4 folder on GitHub.

- Part 1 base and loop-rewired constructions: `topologies.py`, lines 10–55.
- Exact concurrent-flow LP: `concurrent_flow.py`, lines 24–69.
- Upper bound $1/h^*(n,d)$: `concurrent_flow.py`, lines 86–95.
- Part 1 evaluation and figures (one-shot: `python run.py`): `run.py`, lines 51–172. Structural rewiring checks: `test_topologies.py`.
- Part 1 true-optimum yardstick for small n (uses the Part 2 MILP): `optimum_small.py`, lines 26–43.
- Part 2 MILP (Gurobi), whole model: `topology_milp.py`, lines 28–71; variables and objective lines 42–45; degree, conservation and capacity constraints lines 47–62.
- Part 2 hose test matrices: `run.py`, lines 45–89; evaluation against fixed topologies: lines 94–117 (one-shot: `python run.py`).

2 Model

There are n nodes and no other switches. Each node has d outgoing and d incoming directed links of capacity 1. Nodes forward traffic for each other, and a demand may be split over several paths. Given a traffic matrix T, the concurrent flow λ is the largest value such that λT_{ij} can be routed from i to j for *every* pair at the same time without exceeding any link capacity.

3 Part 1: uniform traffic

3.1 Upper bound: what any topology can achieve

Under the uniform matrix $T_{ij} = d/(n-1)$, every node sends d in total, so the total demand is nd, which equals the total link capacity nd.

Each hop costs capacity. A unit of flow that travels h hops occupies one unit of capacity on each of h links. Routing λT_{ij} from i to j therefore uses at least $\lambda T_{ij}\,\mathrm{dist}(i,j)$ capacity. Summing over all pairs:

$$\lambda \frac{d}{n-1} \sum_{i \neq j} \mathrm{dist}(i,j) \;\leq\; nd \quad\Longrightarrow\quad \lambda \;\leq\; \frac{n(n-1)}{\sum_{i \neq j} \mathrm{dist}(i,j)} \;=\; \frac{1}{\bar{h}(G)},$$

where $\bar{h}(G)$ is the average shortest-path hop count. For uniform traffic, small average hop count is necessary for high λ, but not sufficient: routing load must also be balanced. The bound is reached only if (a) all flow follows shortest paths and (b) every link is fully loaded.

No topology has fewer hops than a full d-ary tree. From any node, at most d nodes are 1 hop away, at most d^2 are 2 hops away, and at most d^h are h hops away (the breadth-first argument in the topologies lecture). Filling these layers greedily, $m_h = \min\big(d^h,\, (n-1) - \sum_{k<h} m_k\big)$, gives the smallest possible average hop count $h^*(n,d) = \frac{1}{n-1}\sum_h h\, m_h$, so

$$\lambda \;\le\; 1/\bar{h}(G) \;\le\; 1/h^*(n,d) \qquad \text{for every topology } G.$$

For example, $n = 8, d = 4$ gives layers $(4, 3)$, $h^* = 10/7$, and $\lambda \le 0.7$. Whether a topology can meet this bound exactly depends on (n, d). It can for some: our Part 2 MILP reaches it for $n = 8, d = 4$, and also for $(7, 3)$, $(9, 4)$, $(11, 2)$ and $(11, 5)$ (Table 1). It cannot when n is a "Moore order" $1 + d + \cdots + d^D$ with $d, D \ge 2$ (e.g. $n = 7, d = 2$), because perfect directed Moore graphs do not exist there (Plesník and Znám, 1974). So in general the goal is a construction whose hop counts come close to the bound.

3.2 Construction: loop-rewired generalized de Bruijn graph

We start with the de Bruijn graph from the topologies lecture ("shift left, then append one new digit"). Writing it as arithmetic makes it work for any n:

```
for u in 0 .. n-1:
    for k in 0 .. d-1:
        add link  u -> (d*u + k) mod n
```

Shifting a base-d label left is multiplying by d, and appending digit k is adding k. When $n = d^m$ this is exactly the lecture's de Bruijn graph. Taken mod n, it is the known *generalized de Bruijn digraph* (Reddy, Pradhan and Kuhl, 1980; Imase and Itoh, 1981), which works for any n. Each node has d outgoing links by construction. It also has d incoming links, because as u ranges over $0..n-1$ and k over $0..d-1$, the values $du+k$ are exactly the integers $0, \ldots, dn-1$. Each residue mod n therefore appears exactly d times.

Where the assumption $d \mid n-1$ is used. It gives $n \equiv 1 \pmod{d}$, so $\gcd(d, n) = 1$, which means d is invertible mod n. Then $u \mapsto du \bmod n$ is a permutation, so each node gets its own block of out-neighbours $\{du, \ldots, du + d - 1\}$. Without it the construction degenerates: for $n = 8, d = 4$ ($\gcd = 4$), every even node has the same four out-neighbours $\{0, 1, 2, 3\}$, every odd node has $\{4, \ldots, 7\}$, and 4 of the 32 ports are self-loops (we see the cost in Part 2). The assumption also rules out $n = d^m$, which is why the mod-n generalization is needed in the first place.

Rewiring wasted ports. List the base graph's self-loop nodes in increasing order $s_0, \ldots, s_{q-1}$; remove each $s_i \to s_i$ and add $s_i \to s_{(i+1) \bmod q}$. In our $d \mid n-1$ setting, these are distinct single loops and $q \ge 2$. Each affected node loses one incoming and one outgoing link, then gains one of each from the directed cycle. Thus every node still has exactly d incoming and d outgoing unit-capacity links, with no self-loops. The new edge may be parallel to an existing edge; the capacity matrix records both. This is a deterministic improvement heuristic, *not* a proof of the best topology for arbitrary (n, d). Figure 1 shows both versions at $n = 9, d = 2$.

3.3 Why start from de Bruijn: short paths

In the base graph, every node reaches every other in at most $\lceil \log_d n \rceil$ hops. After m hops from u with digits $k_1, \ldots, k_m$ we are at

$$d^m u + (k_1 d^{m-1} + k_2 d^{m-2} + \cdots + k_m) \pmod{n}.$$

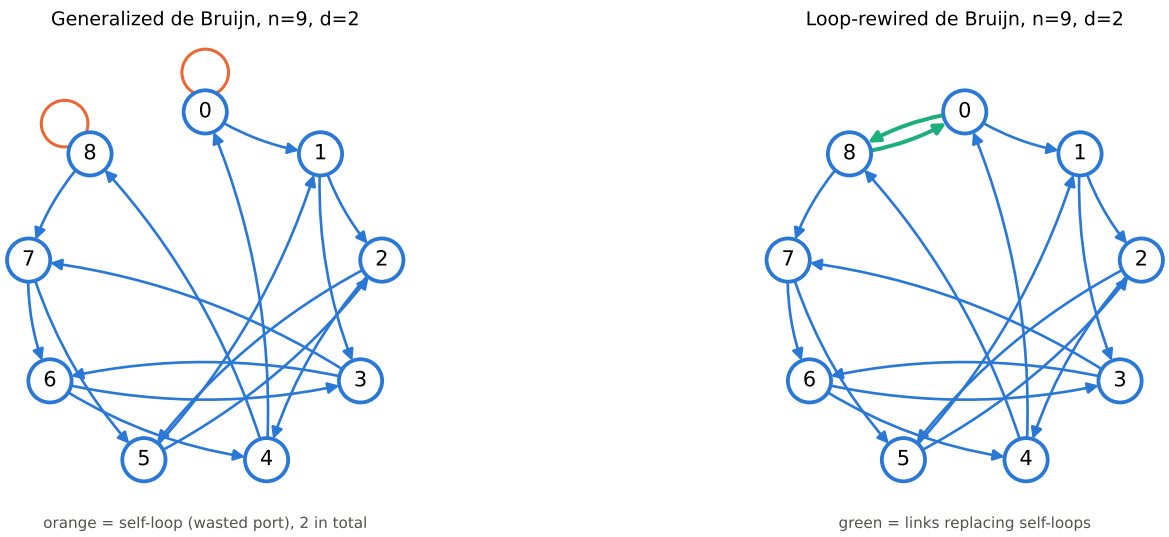


Figure 1: Base (left) and loop-rewired (right) de Bruijn graphs for $n = 9, d = 2$. Orange rings are wasted self-loop ports; green edges replace them.

The bracketed term can be any integer in $0, \ldots, d^m - 1$, the base-d numbers with m digits. Once $d^m \geq n$, these cover every residue mod n, so every target is reachable. The breadth-first lower bound says no topology can reach all n nodes in fewer than about $\log_d n$ hops, so de Bruijn's diameter is within one hop of the smallest possible: it grows logarithmically, as the best possible does. For comparison, the obvious "ring with chords" (simple circulant $u \to u+1, \ldots, u+d$) reaches only d new nodes per hop, so its average hop count grows linearly in n. This proves a bound for the base graph only. Rewiring changes edges, so we measure the new graph's hop counts rather than assume the proof still applies.

3.4 The remaining gap to the bound

As the lecture notes, "regular labels do not guarantee balanced traffic". The base graph misses condition (b) in two ways:

- **Self-loops.** $du + k \equiv u$ can hold. It always holds at $u = 0$ ($k = 0$) and at $u = n - 1$ ($k = d - 1$), and those ports carry nothing.
- **Uneven load.** The graph is not symmetric, so shortest-path routing loads some links more than others. The LP compensates by detouring flow, which costs extra hops.

Rewiring removes the first problem, but does not guarantee balanced load or optimal routing. We therefore measure its remaining gap against both the bound and, where computable, the true optimum. We do not claim exact optimality.

Why not just use the Part 2 MILP? The MILP finds the exact optimum, but it is an exponential-time search, not a construction. With the free Gurobi licence it is only practical for small n (we solved $n \leq 11$), and $(9, 2)$ already needs minutes. Part 1 asks for a rule that works for every n and d. We use the MILP optimum as a yardstick instead (Table 1).

3.5 Evaluation

For each tested (n, d) with $d \mid n-1$ we computed exact λ by a Gurobi LP (one commodity per source, arc flows with conservation and capacity constraints) for the base graph, its rewired variant, and the simple circulant. We compare them with $1/h^*(n, d)$ (Figure 2, Table 1).

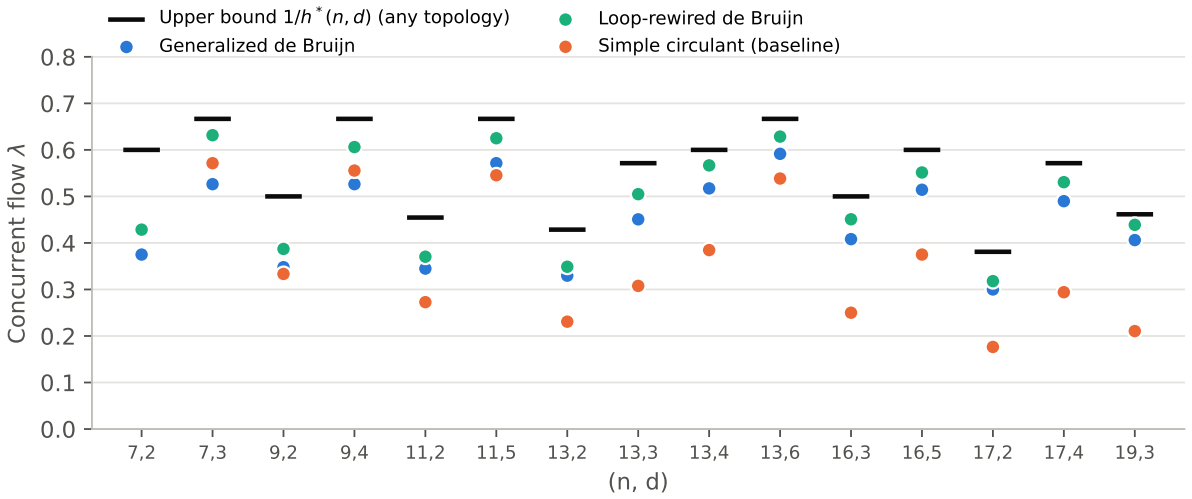


Figure 2: Exact concurrent flow under uniform traffic for both de Bruijn variants and the circulant, versus the upper bound that applies to every topology.

n	d	bound $1/h^*$	true opt.	base λ	rewired λ	rewired % bound	circulant λ
7	2	0.600	0.545	0.375	0.429	71	0.429
7	3	0.667	0.667	0.526	0.632	95	0.571
9	2	0.500	0.488–0.500	0.348	0.387	77	0.333
9	4	0.667	0.667	0.526	0.606	91	0.556
11	2	0.455	0.455	0.345	0.370	81	0.273
11	5	0.667	0.667	0.571	0.625	94	0.545
13	2	0.429	–	0.330	0.349	81	0.231
13	3	0.571	–	0.451	0.505	88	0.308
13	4	0.600	–	0.517	0.567	94	0.385
13	6	0.667	–	0.592	0.629	94	0.538
16	3	0.500	–	0.408	0.451	90	0.250
16	5	0.600	–	0.514	0.552	92	0.375
17	2	0.381	–	0.300	0.318	83	0.176
17	4	0.571	–	0.490	0.531	93	0.294
19	3	0.462	–	0.406	0.439	95	0.211

Table 1: Part 1 results (`part1/results/part1.csv`). "True opt." is the best λ over *all* topologies, from the Part 2 MILP (`part1/optimum_small.py`, `results/part1_optimum.csv`). It is computed only for $n \leq 11$, and for $(9, 2)$ only as a range (time limit). Percentages use unrounded LP and bound values.

Findings.

- Rewiring improves λ in all 15 tested cases and removes all self-loops. The rewired graph reaches 71–95% of the universal bound, versus 63–89% for the base graph. At $(9, 2)$, for example, λ rises from 0.348 to 0.387; this is still below the proven lower bound 0.488 on the optimum.

- Where the true optimum is proven ($n \leq 11$, excluding $(9, 2)$), the rewired graph still falls short: at $(7, 2)$ it gets 0.429 versus 0.545, while at $(7, 3)$ it gets 0.632 versus 0.667. Removing loops alone does not solve the load-balancing problem.
- The rewired graph's measured maximum hop count equals $\lceil \log_d n \rceil$ in all 15 cases, though the base graph proof above does not guarantee this for arbitrary parameters after rewiring.
- It beats the simple circulant in 14 cases and ties at $(7, 2)$. At $(19, 3)$ it gets 0.439 versus 0.211. The measured advantage is consistent with its shorter paths, but routing balance also affects λ.

4 Part 2: arbitrary hose traffic ($n = 8$, $d = 4$)

We present a **mixed-integer linear program**. Given one traffic matrix $T \in \mathcal{T}_{\text{hose}}$, it chooses the topology and the routing together. The topology *adapts* to T because the program is solved again for each new T, as a reconfigurable (e.g. optical circuit-switched) network would pick a new set of circuits.

4.1 Formulation

Let $V = \{0, \ldots, 7\}$, $A = \{(u, v) : u, v \in V,\ u \neq v\}$ and $d = 4$.

Decision variables.

- $x_{uv} \in \{0, 1, \ldots, d\}$ for $(u, v) \in A$: number of unit-capacity links from u to v, i.e. *the topology*. It is integer because a link cannot be half-built. Values above 1 are parallel links.
- $f^s_{uv} \geq 0$ for $s \in V$, $(u, v) \in A$: traffic that originated at source s and crosses link $u \to v$, i.e. *the routing*.
- $\lambda \geq 0$: the fraction of every demand that is delivered.

Objective. max λ.

Constraints.

$$\sum_{v \neq u} x_{uv} = d \quad \forall u \in V, \qquad \sum_{u \neq v} x_{uv} = d \quad \forall v \in V \quad \text{(degree)} \tag{1}$$

$$\sum_{w \neq v} f^s_{vw} - \sum_{u \neq v} f^s_{uv} = \begin{cases} \lambda \sum_t T_{st} & v = s \\ -\lambda T_{sv} & v \neq s \end{cases} \qquad \forall s, v \in V \qquad \text{(conservation)} \tag{2}$$

$$\sum_{s \in V} f^s_{uv} \leq x_{uv} \qquad \forall (u, v) \in A \qquad \text{(capacity)} \tag{3}$$

4.2 Intuition

- With x *fixed*, constraints (2)–(3) are exactly the concurrent-flow LP we use in Part 1 to evaluate a topology. Source s emits λ times its total demand, each destination v absorbs λT_{sv}, and every other node forwards what it receives (net zero). Flow may be split and may take any number of hops.
- Making x a *variable*, limited only by the port budget (1), lets the solver search over every topology with d outgoing and d incoming links at the same time as the routing. The optimum is therefore the largest λ achievable by *any* such topology for this T. That is exactly "the topology that maximizes concurrent flow for the given traffic matrix".

- The model is linear: λT_{st} is a variable times a constant, and capacity compares flows with x. Only x is integer, so it is a MILP. Gurobi proves optimality up to its default tolerance (0.01% MIP gap), and our script asserts this for every solve instead of accepting a time-limited answer.
- Self-loops are excluded because they cannot carry traffic. Parallel links are allowed and necessary: for $T = 4\times$ a permutation, the optimum puts all 4 links on each node's single destination and reaches $\lambda = 1$.
- One commodity per *source* (8 commodities) instead of per pair (56) gives the same λ, because a single-source flow decomposes into paths to each destination. This keeps the model at 56 integer and 449 continuous variables.
- **Upper bound.** We test matrices whose row and column sums are exactly 4, the fully loaded case. Total demand is then 32, which equals total capacity $8 \cdot 4$, so $\lambda \leq 1$, with equality only if every demand travels one hop. The hose set also allows smaller sums. The model handles those unchanged, and λ can then exceed 1. The only exception is $T = 0$, where every λ is feasible, so the code rejects it.

4.3 Evaluation

We solved the MILP for 15 hose matrices, each with every row and column summing to 4 and a zero diagonal:

- a $4\times$ permutation;
- two permutations with weight 2;
- three permutations with real weights $(0.5, 0.3, 0.2) \times 4$, chosen so that no two send to the same destination from the same source;
- a skewed matrix (3 to one partner plus 1 spread uniformly);
- the uniform matrix;
- 10 random dense matrices (uniform random entries scaled by Sinkhorn iteration to row/column sums of 4, seed 422).

Every MILP solution was re-scored by the independent Part 1 LP, and the two λ values agree. As baselines that do *not* adapt, we fixed two topologies and evaluated them on every matrix with the same LP: (i) the MILP-optimal topology for the uniform matrix, chosen once, and (ii) the *original* (not rewired) Part 1 de Bruijn graph for $n = 8, d = 4$ (Figure 3).

Matrix	adaptive (MILP)	fixed: uniform-optimal	fixed: de Bruijn
$4\times$ permutation	1.000	0.542	0.417
2 permutations	1.000	0.625	0.450
3 perms, real weights	0.938	0.676	0.476
skewed	0.803	0.587	0.461
uniform	0.700	0.700	0.500
random (10, mean / range)	0.799 / 0.775–0.812	0.678 / 0.610–0.701	0.509 / 0.487–0.541

Table 2: Part 2 results (`part2/results/part2.csv`).

Findings.

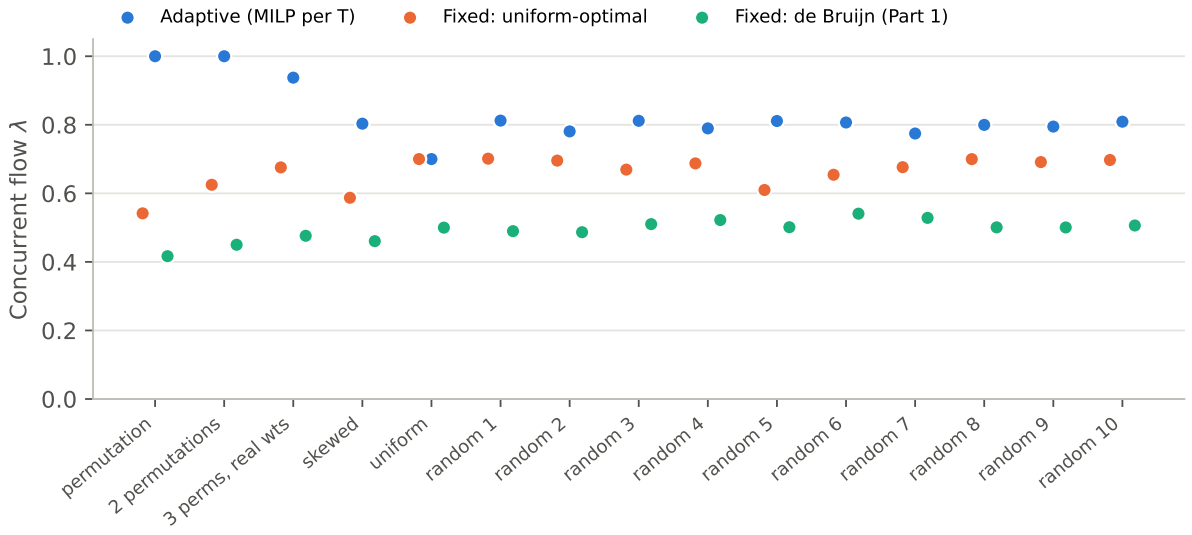


Figure 3: Concurrent flow per hose matrix: topology re-optimized for each T vs. two fixed topologies. Every MILP solve was proven optimal (gap 0 within Gurobi's 0.01% tolerance, checked in code).

- **Integer-structured traffic is served perfectly.** If T is an integer combination of permutations, the MILP builds exactly those direct links (with parallel links where needed) and reaches $\lambda = 1$, the upper bound.
- **Fractional traffic needs some relaying.** For the three-permutation matrix (Figure 4), each demand of 2.0 gets 2 links, but demands of 1.2 and 0.8 can only get 1 link each, since links are whole units and each node has only 4. Part of every 1.2 demand must then travel two hops, using spare capacity on other links (such as the 0.8 links). Two-hop flow costs double capacity, so $\lambda = 0.938 < 1$.
- **Uniform is the hardest of the matrices we tested.** This is an observation from our 15 samples, not a proof over the whole hose set. The MILP reaches $\lambda = 0.7$, exactly Part 1's bound $1/h^*(8, 4)$, so the bound is tight for $n = 8, d = 4$ (the optimal topology has average hop count exactly $10/7$). Random dense matrices land in between (≈ 0.80).
- **Adapting pays off.** A fixed topology can be good for one kind of traffic but not for all. The uniform-optimal topology matches the MILP on uniform traffic but delivers only about half of a permutation (0.542). On random matrices, adapting gives 18% more λ on average. De Bruijn is worst here because $n = 8, d = 4$ violates Part 1's assumption $d \mid n - 1$: $\gcd(4, 8) = 4$, so the construction degenerates (Section 3.2). Every even node has the same four out-neighbours, and 4 of the 32 ports are self-loops.

5 AI use

Claude (Claude Code) was used interactively during the initial implementation for conceptual guidance (concurrent flow, the hop-count bound, mapping lecture slides to the problem), proposing and comparing candidate topologies, and drafting the Gurobi LP/MILP code, evaluation scripts, figures, and report text. Codex assisted with the later loop-rewiring implementation, structural tests, LP reruns, figure regeneration, and this report update. The code was validated against hand-computed cases (complete

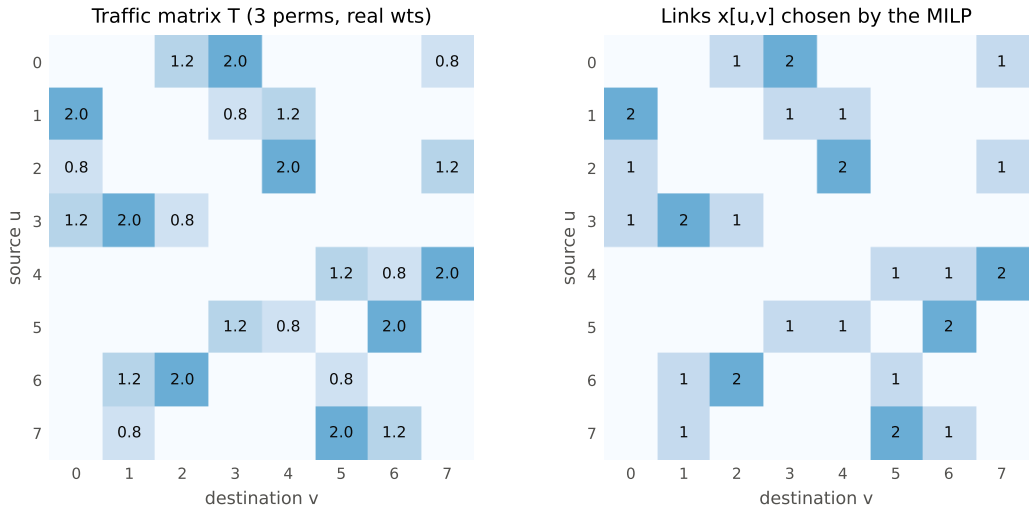


Figure 4: The topology follows the demand: three-permutation matrix (left) and the link counts the MILP chooses for it (right), $\lambda = 0.938$.

graph, directed ring, permutation, uniform bound) and the measured data in the repository. Decisions and rationale are logged in `DECISIONS.md`, and AI use in `AI_USAGE.md`.